STEP-BY-STEP GUIDE

Playwright + TypeScript Automation Framework

Building a Test Automation Framework with Playwright and TypeScript

Playwright · TypeScript · Node.js · Page Object Model · Fixtures · Web · API · DB · Allure · Playwright MCP / CLI · GitHub Actions · Docker · Jenkins

1. **Setup** — Node · VS Code · Playwright
2. **Structure** — Folders & config
3. **Assets** — Data · API · Prompts
4. **Generate** — AI-built tests
5. **Run** — Tagged npm scripts
6. **Ship** — GitHub · Docker · Jenkins

13 steps • Web · API · DB testing • AI-assisted with Claude Code · Copilot · CommandCode · OpenCode

Contents

Introduction

How to Read This Document

Glossary: Key Terms Used in This Guide

Prerequisites (Manual Actions/Steps)

- Install
- Install Playwright Using the VS Code Terminal
- Install Other Dependencies

Step by Step: Building the Framework

01 Create the Project Folder Structure
02 Add or Update Configuration Files
03 Add Test Data Files
04 Add API Endpoints and Schemas Under the api/ Folder
05 Create the Custom Reporter and Place It in the utils Folder
06 Place Prompt Files in the prompts Folder
07 Create Helper & Utility Files
08 Generate Tests Using the Prompts Stored in the prompts Folder
09 Update package.json With Custom Test Commands
10 Push the Project to GitHub

CI / CD (Continuous Integration & Delivery)

11 Playwright GitHub Actions Workflow
12 Docker
13 Jenkins Pipeline

01 Introduction

This guide walks you step by step through setting up a complete automated testing framework using Playwright and TypeScript, and using an AI coding agent to help generate configuration, utilities, and tests. It uses a demo project called `OpenCart_Playwright` as the running example, but the same approach can be adapted to other projects.

The original instructions, commands, configuration values, and overall flow from the source document are preserved. The AI prompts have been refined for clarity, consistency, and better results when used with an AI coding agent.

02 How to Read This Document

- 🤖 **PROMPT** sections contain text you can copy and paste into your AI coding agent, such as Claude Code, GitHub Copilot, CommandCode, or OpenCode.
- 💻 **Code** sections contain file contents or commands that you type or paste directly into your project.
- ⚠️ **Note** sections call out important warnings or clarifications.
- 💡 **Why this matters** sections explain the purpose behind a step so you understand not only how to do it, but why it is needed.

03 Glossary: Key Terms Used in This Guide

If you're new to test automation, skim this section first. You can always come back to it as terms come up later in the guide.

TERM	WHAT IT MEANS
Node.js	A program that lets you run JavaScript/TypeScript code outside a web browser. It is required to install and run Playwright.
npm	"Node Package Manager." The tool installed together with Node.js that is used to download and install code libraries, such as the ones used in this framework.
Playwright	An open-source tool by Microsoft that automates web browsers: it can open pages, click buttons, fill forms, and check results, which is exactly what automated tests need to do.
TypeScript	A version of JavaScript that adds type checking, which helps catch mistakes before you run your code. Playwright supports writing tests in TypeScript or plain JavaScript; this guide uses TypeScript.
VS Code	Visual Studio Code, a free, popular code editor used to write and run the project files in this guide.
Coding agent / AI agent	An AI assistant, such as Claude Code, GitHub Copilot, CommandCode, or OpenCode, that can write or edit code when you give it a written instruction (a "prompt").
Prompt	The instructions you type or paste to an AI coding agent describing what you want it to build or change.
Page Object Model (POM)	A common way of organizing test code where each web page, such as Login or Cart, has its own file describing how to interact with it. These files live in the `pages/` folder.
Fixtures	Reusable setup and clean-up code that multiple tests can share, for example logging in before a test starts. These live in the `fixtures/` folder.
Allure Report	A tool that turns raw test results into an easy-to-read HTML report with charts, history, and screenshots.
CI/CD	"Continuous Integration / Continuous Deployment": automatically building and testing your code whenever it changes, typically using tools like GitHub Actions or Jenkins.
MCP (Model Context Protocol)	A way for AI agents to directly use tools such as a web browser through Playwright, allowing the AI to interact with a real page while generating tests.
Docker	A tool that packages your project and everything it needs to run into a portable "container," so it behaves consistently across environments.
`.env` **file**	A plain-text file that stores configuration values and settings, such as URLs and test credentials, separately from your code.

TERM	WHAT IT MEANS
`.gitignore`	A file that tells Git which files or folders to exclude from version control, such as `node_modules` or generated reports.

04 Prerequisites (Manual Actions/Steps)

Before you start building the framework, install the following tools on your computer. Each one plays a specific role in the setup.

Install

1. **Node.js (latest)** — https://nodejs.org/en — required to run Playwright and npm commands.
2. **Visual Studio Code (latest)** — https://code.visualstudio.com/ — the code editor you'll use throughout this guide.
3. **Coding Agent** — Claude Code / GitHub Copilot / CommandCode / OpenCode or any other AI coding agent. This is the AI assistant that will generate files for you whenever a step includes a prompt.

💡 **Why this matters:** Node.js is the engine that runs everything else in this guide, VS Code is where you'll view and edit files, and the coding agent is what turns the written prompts into real code.

Install Playwright Using the VS Code Terminal

Open a terminal inside VS Code:

Terminal → New Terminal

Run:

```
› Terminal
npm init playwright@latest
```

This command downloads Playwright and walks you through a short setup wizard.

Choose the following options when prompted:

#	QUESTION	ANSWER
1	Do you want to use TypeScript or JavaScript?	**TypeScript**
2	Where to put your end-to-end tests?	**tests**
3	Add a GitHub Actions workflow? (Y/n)	**true**
4	Install Playwright browsers?	**true**

💡 **Why this matters:** This wizard creates the basic Playwright project skeleton for you — the config file, an example test, and, if selected, a ready-made GitHub Actions workflow file — so you don't have to build it from scratch.

Install Other Dependencies

Create a file named `env_setup.bat` in the project root.

This file contains the commands below so anyone on the team can install the required dependencies in one go instead of typing each command separately.

Commands — place inside `env_setup.bat`

```batch
npm install dotenv
npm install @faker-js/faker
npm install luxon
npm install ajv csv-parse xlsx
npm install @axe-core/playwright
npm install allure-playwright
npm install -D @types/node
npx playwright install
npm install mysql2
```

Quick Reference — What Each Package Is For

#	PACKAGE	PURPOSE
1	`dotenv`	Loads settings from your `.env` file into your code.
2	`@faker-js/faker`	Generates realistic fake test data such as names, emails, and addresses.
3	`luxon`	Handles dates and times.
4	`ajv` , `csv-parse` , `xlsx`	Validate JSON data and read CSV/Excel test-data files.
5	`@axe-core/playwright`	Adds accessibility checks to your tests.
6	`allure-playwright`	Produces the Allure report used later in this guide.
7	`@types/node`	Gives TypeScript awareness of Node.js built-in features.
8	`playwright install`	Downloads the actual browsers Playwright will control.
9	`mysql2`	Lets your tests connect to a MySQL database.

Run the `env_setup.bat` file from the terminal.

⚠️ **Note:** You only need to run `env_setup.bat` once per machine, or again later if new dependencies are added to the list.

05 Step by Step: Building the Framework

With the prerequisites installed, follow these 13 steps in order.

- Steps marked **Manual** are performed by you.
- Steps marked **Using Prompt** can be handed to your AI coding agent.

01 STEP 1 Create the Project Folder Structure

MODE ✋ Manual *or* 🤖 Using Prompt

💡 **Why this matters:** A consistent folder structure keeps configuration, test data, page objects, and reports separated and easy to find as the project grows.

🤖 PROMPT Copy/Paste This to Your AI Coding Agent

✦ **Prompt — copy / paste to your AI coding agent**

```
Create the required folder and file structure for the project root `OpenCart_Playwright`.
Use the target structure shown in the supplied project context/document.

Requirements:
- Create only the required folders and placeholder files.
- Preserve all existing project files unless they must be created or updated.
- Do not delete or overwrite existing implementation files unnecessarily.
- Keep the structure organized for Playwright + TypeScript automation covering Web, API,
and DB testing.
- Use the existing project naming conventions.
- After creating the structure, show the final directory tree and briefly identify any
files that were newly created.

Target structure:
OpenCart_Playwright/
├── .env
├── .gitignore
├── api/
│   ├── endpoints/
│   └── schemas/
├── custom-report/
├── fixtures/
├── pages/
```

```
├── prompts/
├── tests/
│   ├── web/
│   ├── api/
│   └── db/
├── testdata/
├── utils/
├── reports/
```

02

STEP 2

Add or Update Configuration Files

MODE ✋ Manual

Three configuration files control how the framework behaves:

FILE	PURPOSE
`playwright.config.ts`	Playwright test-runner settings
`tsconfig.json`	TypeScript settings
`.env`	Environment values and secrets

Create or update the following configuration files in the project:

- `playwright.config.ts`
- `tsconfig.json`
- `.env`

⚠️ **Note:** Use the exact configuration values and settings provided in this guide.

💻 `playwright.config.ts`

This file tells Playwright how to run your tests — timeouts, browsers, retries, reporters, traces, screenshots, videos, and other execution settings.

❯ TypeScript

```typescript
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  timeout: 30 * 1000, //30000 ms(30 secs)
  testDir: "./tests",
  fullyParallel: true,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 1 : undefined,
```

```
  reporter: [
    ["list"], // Detailed console output
    //['line'], // One-line progress output
    //['dot'], // Minimal console output
    ["html", { open: "never", outputFolder: "reports" }], // HTML Report
    //['json', { outputFile: 'reports/results.json' }], // JSON Report
    ["junit", { outputFile: "reports/results.xml" }], // JUnit XML Report
    ["./utils/CustomReporter.ts"], // Custom reporter
    ["allure-playwright", { outputFolder: "allure-results" }], // Allure Report
  ],

  use: {
    trace: "on-first-retry",
    screenshot: "only-on-failure",
    video: "retain-on-failure",
    headless: false,
    viewport: { width: 1280, height: 720 }, // Set default viewport size for consistency
    ignoreHTTPSErrors: true, // Ignore SSL errors if necessary
    permissions: ["geolocation"], // Set necessary permissions for geolocation-based tests
  },

  grep: /@master/,

  projects: [
    {
      name: "chromium",
      use: { ...devices["Desktop Chrome"] },
    },
    /*
    {
      name: 'firefox',
      use: { ...devices['Desktop Firefox'] },
    },
    {
      name: 'webkit',
      use: { ...devices['Desktop Safari'] },
    }
    */
  ],
});
```

💻 `tsconfig.json`

This file configures how TypeScript compiles your code and defines shortcut import paths such as `@utils/` and `@pages/`.

```
❯ JSON
```

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "Node16",
    "lib": ["ES2022", "DOM"],
    "moduleResolution": "node16",
    "types": ["node"],
    "strict": true,
```

```
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true,
    "resolveJsonModule": true,
    "isolatedModules": false,
    "outDir": "dist",
    "ignoreDeprecations": "6.0"
  },
  "include": ["playwright.config.ts"],
  "exclude": ["node_modules"]
}
```

💻 `.env`

This file stores URLs, login credentials, and other settings your tests need separately from your code.

> ⚠️ **Note:** Values shown below, including sample credentials, are demo/test values used for this OpenCart practice project. Replace them with your own project's values and do not commit real secrets to source control.

`.env`

```
# 🌍 ENVIRONMENT
APP_ENV=qa # qa | prod | dev

### 🌐 WEB APP CONFIGURATION ###

# 🔗 WEB APP URL
WEB_APP_URL=http://localhost/opencart/upload/
#WEB_APP_URL=https://tutorialsninja.com/demo/
#WEB_APP_URL=https://awesomeqa.com/ui/

# 🔐 WEB APP LOGIN CREDENTIALS (CREATE YOUR OWN CREDENTIALS)
APP_EMAIL=YOUR_APP_EMAIL
APP_PASSWORD=YOUR_APP_PASSWORD

# 🛒 PRODUCT DETAILS
PRODUCT_NAME=MacBook
PRODUCT_QUANTITY=1
TOTAL_PRICE=$602.00

### ⚡ API CONFIGURATION ###

# 🔗 API BASE URL
API_BASE_URL=https://fakestoreapi.com

# 👤 API USER DETAILS
USERNAME=mor_2314
PASSWORD=83r5^_
USER_ID=1

# 🛍️ API PRODUCT DETAILS
PRODUCT_ID=1
CART_ID=1
```

```
LIMIT=3

# 🗓️ DATE RANGE
START_DATE=2019-12-10
END_DATE=2020-10-10

# 🔑 BASIC AUTHENTICATION CREDENTIALS (IF REQUIRED)
#BASIC_AUTH_USERNAME=YOUR_USERNAME
#BASIC_AUTH_PASSWORD=YOUR_PASSWORD

# 🔐 API KEY (IF REQUIRED)
#API_KEY=YOUR_API_KEY

# 🎟️ ACCESS TOKEN (IF REQUIRED)
#ACCESS_TOKEN=YOUR_PERSONAL_ACCESS_TOKEN

# 🔄 OAUTH2 CONFIGURATION (IF REQUIRED)
#CLIENT_ID=YOUR_CLIENT_ID
#CLIENT_SECRET=YOUR_CLIENT_SECRET
#AUTHORIZATION_CODE=YOUR_AUTHORIZATION_CODE
#TOKEN_URL=YOUR_TOKEN_URL

### 🗄️ DATABASE CONFIGURATION ###

# 🗄️ DATABASE CONNECTION
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=openshop
DB_PORT=3306

# 🧹 ADMIN APP LOGIN CREDENTIALS
ADMIN_USERNAME=admin
ADMIN_PASSWORD=admin
```

🔒 **Important:** For your project, keep the actual `.env` file in `.gitignore` because it contains credentials. And keep only placeholder values in `.env.example`

03 STEP 3 Add Test Data Files

MODE ✋ Manually

💡 **Why this matters:** Keeping test data, such as login credentials or product information, in separate files instead of hard-coding it into tests makes tests reusable and easy to update.

Place test data files inside the `testdata/` folder.

```Text
testdata/
├── opencart_logindata.json
├── opencart_logindata.csv
└── opencart_logindata.xlsx
```

04

STEP 4

Add API Endpoints and Schemas Under the api/ Folder

MODE ✋ Manually

💡 **Why this matters:** Separating API route definitions (`endpoints/`) from the JSON "shapes" you expect responses to match (`schemas/`) keeps API tests organized and makes it easy to validate responses automatically.

```Text
api/
├── endpoints/
│   └── routes.ts                # Centralized endpoint URL constants
└── schemas/
    ├── product_api_schema.json   # JSON Schema for product responses
    ├── cart_api_schema.json      # JSON Schema for cart responses
    └── user_api_schema.json      # JSON Schema for user responses
```

05

STEP 5

Create the Custom Reporter and Place It in the utils Folder

MODE ✋ Manually

💡 **Why this matters:** The custom reporter referenced in `playwright.config.ts` needs an actual file to run — this is where it lives.

```Text
utils/
└── CustomReporter.ts
```

06

STEP 6

Place Prompt Files in the prompts Folder

MODE ✋ Manually

These are the written instructions you will later provide to your AI coding agent in Step 8 to generate the actual tests.

Add the following files:

1. `opencart_web_test_prompts.md`
2. `fakestore_api_test_prompts.md`
3. `db_test_prompt.md`
4. `utilitis_prompt.md`

```Text
promtps/
|── opencart_web_test_prompts.md
|── fakestore_api_test_prompts.md
|── db_test_prompt.md
└── utilitis_prompt.md
```

07

STEP 7

Create Helper & Utility Files

MODE ✋ Manually *or* 🤖 Using Prompt

💡 **Why this matters:** Helper and utility files, such as data readers, generators, DB clients, and common helpers, provide reusable building blocks that your tests and page objects can call instead of repeating the same logic everywhere.

Supply the `utilitis_prompt.md` file to the AI agent and give it the prompt below.

🤖 PROMPT Copy/Paste This to Your AI Coding Agent

✦ **Prompt — copy / paste to your AI coding agent**

```
Read and follow all instructions in `prompts/utilitis_prompt.md`.
Create or update the required helper and utility files under the `utils/` folder.

Requirements:
- Follow every requirement defined in `utilitis_prompt.md`.
- Reuse existing utilities when appropriate instead of creating duplicate functionality.
- Keep the implementation compatible with Playwright + TypeScript.
```

```
- Use the existing project structure, aliases, naming conventions, and dependencies.
- Do not modify unrelated files.
- Do not create unnecessary utilities.
- If a required utility already exists, update it only when necessary.
- After implementation, list the utility files created or updated and briefly explain their purpose.
```

```
Text
utils/
└── dataGenerator.ts
└── DataReader.ts
└── dbClient.ts
└── helper.ts
```

08 STEP 8 Generate Tests Using the Prompts Stored in the prompts Folder

You can give the AI coding agent each prompt one at a time, or all of them at once.

> ⚠️ **Note:** Make sure you have installed **Playwright MCP** or the **Playwright CLI** before proceeding further. Clearly state in the prompt which mechanism the agent should use: **MCP** or **CLI**.

- Refer to the Playwright CLI Documentation: https://playwright.dev/agent-cli/introduction
- Installation instructions: https://playwright.dev/agent-cli/installation

```
Terminal
# Installation
npm install -g @playwright/cli@latest
playwright-cli install
playwright-cli install --skills

# Verify
playwright-cli --version
```

Attach `playwright_mcp_context.md`, then provide the test prompts from the following files:

1. `opencart_web_test_prompts.md`
2. `fakestore_api_test_prompts.md`
3. `db_test_prompt.md`

🌐 Prompt 1 — Web

PROMPT Copy/Paste This to Your AI Coding Agent

Prompt — copy / paste to your AI coding agent

```
Use the supplied Playwright MCP/CLI setup and the attached `playwright_mcp_context.md` as
the implementation context.

Read and follow the web test requirements in `prompts/opencart_web_test_prompts.md`.
Generate the required Playwright + TypeScript web tests.

Requirements:
- Follow the supplied test prompt exactly.
- Use Playwright MCP or Playwright CLI as specified for this task.
- Inspect the application when browser interaction is required.
- Follow the framework structure and conventions defined in the supplied context.
- Use Page Object Model and fixtures where required by the framework/context.
- Reuse existing utilities, test data, configuration, and fixtures whenever applicable.
- Do not duplicate existing framework functionality.
- Do not modify unrelated files.
- Add appropriate test tags and assertions based on the supplied test requirements.
- Keep tests maintainable, readable, and suitable for repeated execution.
- After implementation, list the files created or updated and explain what was implemented.
```

🔌 Prompt 2 — API

PROMPT Copy/Paste This to Your AI Coding Agent

Prompt — copy / paste to your AI coding agent

```
Use the supplied Playwright MCP/CLI setup and the attached `playwright_mcp_context.md` as
the implementation context.

Read and follow the API test requirements in `prompts/fakestore_api_test_prompts.md`.
Generate the required Playwright + TypeScript API tests.

Requirements:
- Follow the supplied test prompt exactly.
- Use the existing API configuration, endpoint definitions, schemas, utilities, and
fixtures where applicable.
- Reuse existing framework components instead of duplicating functionality.
- Validate HTTP status codes, response data, and schemas wherever required by the supplied
test prompt.
- Keep API tests independent unless the supplied test requirements explicitly define a
dependency or sequence.
- Do not hard-code environment-specific values that belong in `.env` or test-data files.
- Do not modify unrelated files.
- Add appropriate test tags.
- After implementation, list the files created or updated and briefly explain the API
coverage implemented.
```

Prompt 3 — DB

PROMPT Copy/Paste This to Your AI Coding Agent

Prompt — copy / paste to your AI coding agent

```
Use the supplied Playwright MCP/CLI setup and the attached `playwright_mcp_context.md` as
the implementation context.

Read and follow the database test requirements in `prompts/db_test_prompt.md`.
Generate the required Playwright + TypeScript database tests.

Requirements:
- Follow the supplied database test prompt exactly.
- Use the project's existing DB configuration and `mysql2`-based database utilities where
applicable.
- Reuse existing DB client/helper functionality instead of creating duplicate
implementations.
- Keep database credentials and environment-specific values in `.env`.
- Validate the database results required by the supplied test prompt.
- Ensure database connections are properly handled and released.
- Do not modify unrelated files.
- Add appropriate test tags.
- After implementation, list the files created or updated and briefly explain the database
coverage implemented.
```

⚠️ **Note:** Page Object classes (`pages/`) and fixtures (`fixtures/`) are not created manually — the coding agent generates them automatically as it builds each test, based on the context supplied in `playwright_mcp_context.md` .

09

STEP 9

Update package.json With Custom Test Commands

MODE ✋ Manually

💡 **Why this matters:** Custom npm scripts let you or your CI pipeline run a specific group of tests — such as only `@sanity` tests — with one short command instead of typing the full Playwright command every time.

Update `package.json` to add the following custom scripts for running tagged test suites:

JSON

```json
"scripts": {
  "test:api": "playwright test --grep @api",
  "test:web": "playwright test --grep @web",
```

```
  "test:master": "playwright test --grep @master",
  "test:sanity": "playwright test --grep @sanity",
  "test:regression": "playwright test --grep @regression",
  "test:e2e": "playwright test --grep @e2e",
  "test:datadriven": "playwright test --grep @datadriven",
  "test:master:headed": "playwright test --grep @sanity --headed",
  "test:sanity:debug": "playwright test --grep @sanity --debug"
}
```

Example Commands to Run the Tests

COMMAND	DESCRIPTION
`npm run test:api`	Run tests tagged with `@api`
`npm run test:web`	Run tests tagged with `@web`
`npm run test:master`	Run tests tagged with `@master`
`npm run test:sanity`	Run tests tagged with `@sanity`
`npm run test:regression`	Run tests tagged with `@regression`
`npm run test:e2e`	Run tests tagged with `@e2e`
`npm run test:datadriven`	Run tests tagged with `@datadriven`
`npm run test:master:headed`	Run tests tagged with `@master` in headed mode
`npm run test:sanity:debug`	Run tests tagged with `@sanity` in debug mode

Run the Tests

```
❯ Terminal

npm run test:master
```

Generate and View the Allure Report

The Allure reporter was registered in `playwright.config.ts` in Step 2.

```
❯ Terminal

allure generate ./allure-results -o ./allure-report --clean
allure open ./allure-report
```

10

STEP 10

Push the Project to GitHub

MODE Using Prompt *or* Manually

Git Workflow - Using git commands

The most commonly used commands are:

Terminal

```
git init
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
git status
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/USERNAME/REPOSITORY.git
git push -u origin master
git pull
```

PROMPT Copy/Paste This to Your AI Coding Agent

Prompt — copy / paste to your AI coding agent

```
Add, commit, and push the latest project changes to the configured GitHub repository.

Follow these steps:
1. Review the current Git status and identify all project changes.
2. Ensure appropriate files are staged. Do not commit secrets, `.env` files, generated
files, dependencies, or other files excluded by `.gitignore`.
3. Stage the required changes.
4. Create a clear and descriptive commit message based on the actual changes.
5. Commit the changes to the current branch.
6. Push the commit to the configured GitHub remote repository.
7. If there are no changes to commit, do not create an empty commit. Report that the
working tree is clean.
8. If the push fails, report the exact reason and do not claim that the push was
successful.

After completion, report:
Commit message,Branch,Push result (Successful or Failed or No changes to push)
```

06 CI / CD (Continuous Integration & Delivery)

11 STEP 11 Playwright GitHub Actions Workflow

MODE ✋ Manually

💡 **Why this matters:** A GitHub Actions workflow automatically runs your tests, for example on every push or pull request, and stores the resulting reports without anyone needing to run them manually.

.github\workflows\playwright.yml

```yaml
#To run manually: go to your repo → Actions → Playwright Tests → Run workflow → choose your parameters → Run.

name: 🎭 Playwright Tests

on:
  push:
    branches: [master]
  pull_request:
    branches: [master]
  workflow_dispatch:
    inputs:
      test_suite:
        description: "Select test suite"
        type: choice
        options:
          - test:master
          - test:sanity
          - test:regression
          - test:e2e
          - test:api
          - test:web
          - test:datadriven
        default: "test:e2e"
      browser:
        description: "Select browser"
        type: choice
        options:
          - chromium
          - firefox
          - webkit
        default: "chromium"
      mode:
        description: "Run mode"
        type: choice
        options:
          - headless
```

```
          - headed
        default: "headless"

jobs:
  test:
    timeout-minutes: 60
    runs-on: windows-latest

    steps:
      - name: 🛎️ Checkout Repository
        uses: actions/checkout@v4

      - name: 🟢 Set up Node.js
        uses: actions/setup-node@v4
        with:
          node-version: lts/*

      - name: 📦 Install Dependencies
        run: npm ci

      - name: 🎭 Install Playwright Browsers
        run: npx playwright install --with-deps

      - name: ⚙️ Create .env File
        run: Copy-Item .env.example .env

      - name: 🧪 Run Playwright Tests
        run: |
          $suite = "${{ github.event.inputs.test_suite || 'test:e2e' }}"
          $browser = "${{ github.event.inputs.browser || 'chromium' }}"
          $mode = "${{ github.event.inputs.mode || 'headless' }}"
          $tag = $suite -replace 'test:', ''
          $browserFlag = "--project=$browser"
          $modeFlag = if ($mode -eq 'headed') { '--headed' } else { '' }
          npx playwright test --grep "@$tag" $browserFlag $modeFlag --reporter=dot --output=test-
results
        env:
          CI: "1"

      - name: 📊 Upload Playwright HTML Report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: playwright-report
          path: reports/
          retention-days: 30

      - name: 📋 Upload Custom Report
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: custom-report
          path: custom-report/
          retention-days: 30

      - name: 📈 Upload Allure Results
        if: always()
        uses: actions/upload-artifact@v4
```

```
      with:
        name: allure-results
        path: allure-results/
        retention-days: 30
```

12

STEP 12

Docker

MODE 🖐 Manually

💡 **Why this matters:** Docker lets you run the entire test suite in an identical, self-contained environment.

⚠️ **Note** Follow the Docker Setup documentation for detailed steps.

Prerequisites

Make sure you have Docker Desktop installed:

- **Docker Desktop** — Download for Windows | Download for Mac

After installing, launch Docker Desktop and wait for the whale icon to stop animating.

Files to be created

FILE	WHAT IT DOES
`Dockerfile`	Defines the Docker image — starts from `playwright:v1.62.0-noble`, installs npm dependencies, copies source code, sets up the entrypoint.
`docker-compose.yml`	Orchestrates the container — tells Docker which image to build, which files to mount, which environment variables to use, and which test suite to run.
`docker-entrypoint.sh`	A smart script that translates a simple suite name (like `sanity` or `api`) into the correct `npm run` command. You never need to edit this.
`.dockerignore`	Tells Docker which files to skip when building the image (e.g., `node_modules`, `.env`, old reports). Keeps the image small and secure.
`.env.example`	A template showing all available environment variables. Copy it to `.env` and fill in your values.

Dockerfile

```
# Base image with Playwright v1.62.0, Node.js, and system dependencies
FROM mcr.microsoft.com/playwright:v1.62.0-noble
```

```
# Set working directory
WORKDIR /app

# Copy dependency manifests first (leverages Docker layer caching)
COPY package*.json ./

# Install project dependencies (clean install — respects package-lock.json)
RUN npm ci

# Copy the rest of the project source
COPY . .

# Create output directories for reports and test artifacts
RUN mkdir -p /app/reports /app/custom-report /app/allure-results /app/allure-report /app/test-
results

# Enable CI mode — Playwright config uses this to adjust retries, workers, headless
ENV CI=true

# Point to the Playwright browsers bundled in the base image
ENV PLAYWRIGHT_BROWSERS_PATH=/ms-playwright

# Make the entrypoint script executable
RUN chmod +x /app/docker-entrypoint.sh

# Use the entrypoint so the suite name argument is handled automatically
ENTRYPOINT ["/app/docker-entrypoint.sh"]

# Default: run all tests
CMD ["all"]
```

docker-compose.yml

```
services:
  playwright-tests:
    build:
      context: .
      dockerfile: Dockerfile
    image: playwright-opencart
    container_name: playwright-opencart-tests
    working_dir: /app
    env_file:
      - .env
    environment:
      CI: "1"
    volumes:
      # Mount the project root so code changes are reflected without rebuilds
      - .:/app
      # Preserve the container's node_modules (bind mount would shadow it)
      - /app/node_modules
      # Persist reports and artifacts on the host
      - ./reports:/app/reports
      - ./allure-results:/app/allure-results
      - ./allure-report:/app/allure-report
      - ./custom-report:/app/custom-report
      - ./test-results:/app/test-results
```

```
    # Default suite — override via command-line argument
    # docker compose run --rm playwright-tests api
    command: [ "all" ]
    stdin_open: true
    tty: true
```

docker-entrypoint.sh

```
set -e

SUITE="${1:-all}"

case "$SUITE" in
  all)
    shift 2>/dev/null || true
    echo "⟹ Running ALL tests (args: $@)"
    exec npx playwright test "$@"
    ;;
  sanity|regression|web|api|master|e2e|datadriven)
    shift
    echo "⟹ Running '$SUITE' tests (args: $@)"
    exec npm run "test:${SUITE}" -- "$@"
    ;;
  *)
    echo "Error: Unknown test suite '$SUITE'"
    echo ""
    echo "Usage: $0 {all|sanity|regression|web|api|master|e2e|datadriven} [playwright-args...]"
    echo ""
    echo "Available test suites:"
    echo "  all          Run all tests (default)"
    echo "  sanity       Run sanity tests (@sanity)"
    echo "  regression   Run regression tests (@regression)"
    echo "  web          Run web tests (@web)"
    echo "  api          Run API tests (@api)"
    echo "  master       Run master tests (@master)"
    echo "  e2e          Run e2e tests (@e2e)"
    echo "  datadriven   Run data-driven tests (@datadriven)"
    exit 1
    ;;
esac
```

.dockerignore

```
# Dependencies
node_modules

# Environment & secrets
.env
.env.example

# Test artifacts (bind-mounted at runtime, not part of the image)
test-results
reports
allure-results
```

```
allure-report
custom-report
playwright-report

# Documentation & prompts (not needed at runtime)
docs/
prompts/
*.md

# CI/CD & platform-specific config
Jenkinsfile
env_setup.bat
docker-compose.yml

# Playwright / IDE cache & config
.playwright/
.playwright-cli/
.playwright-mcp/
.vscode/

# Version control
.git
.gitignore
.github
.claude
.commandcode
```

13

STEP 13

Jenkins Pipeline

MODE ✋ Manually

💡 **Why this matters:** Jenkins is another common way, alongside or instead of GitHub Actions, to automatically run your test suite as part of a build pipeline, especially in company/enterprise environments.

Pipeline Stages:

1. **Checkout:** Checks out the source code from the repository.
2. **Install Dependencies:** Installs project dependencies using `npm ci`.
3. **Install Playwright Browsers & Dependencies:** Installs Playwright browsers and copies the `.env.example` file to `.env` if it doesn't exist.
4. **Run Playwright Tests:** Executes the selected Playwright test suite with the specified browser and mode (headless or headed).
5. **Post Actions:** Archives test reports, publishes JUnit and Allure reports, and cleans up `allure-results` on success.
6. **Email Notification:** Sends an email notification with the build status and links to the reports.
7. **Parameters:** Allows users to select the test suite, browser, and mode for the test execution.

Jenkinsfile (Windows)

```
pipeline {
    agent any

    tools {
        git 'Default'
    }

    options { timestamps() }

    parameters {
        choice(
            name: 'TEST_SUITE',
            choices: ['test:master', 'test:sanity', 'test:regression', 'test:e2e', 'test:api',
'test:web', 'test:datadriven'],
            description: 'Select the test suite to run'
        )
        choice(
            name: 'BROWSER',
            choices: ['chromium', 'firefox', 'webkit'],
            description: 'Select the browser for tests'
        )
        choice(
            name: 'MODE',
            choices: ['headless', 'headed'],
            description: 'Run tests in headless or headed mode'
        )
    }

    environment {
        NODE_ENV = 'test'
        CI = '1'
    }

    stages {
        stage('🔄 Checkout') {
            steps {
                checkout scm
            }
        }

        stage('📦 Install Dependencies') {
            steps {
                powershell 'npm ci'
            }
        }

        stage('🤖 Install Playwright Browsers & Dependencies') {
            steps {
                powershell 'npx playwright install --with-deps'
                powershell 'Copy-Item .env.example .env -ErrorAction SilentlyContinue'
            }
        }

        stage('▶ Run Playwright Tests') {
            steps {
```

```
                script {
                    def suite = params.TEST_SUITE
                    def browserFlag = "--project=${params.BROWSER}"
                    def modeFlag = params.MODE == 'headed' ? '--headed' : ''
                    powershell "npm run ${suite} -- ${browserFlag} ${modeFlag}"
                }
            }
        }
    }

    post {
        always {
            archiveArtifacts artifacts: 'reports/**,custom-report/**,allure-results/**',
allowEmptyArchive: true
            junit testResults: 'reports/results.xml', allowEmptyResults: true
            allure includeProperties: false, results: [[path: 'allure-results']]

            publishHTML([
                allowMissing: true,
                alwaysLinkToLastBuild: true,
                keepAll: true,
                reportDir: 'reports',
                reportFiles: 'index.html',
                reportName: 'Playwright HTML Report'
            ])

            publishHTML([
                allowMissing: true,
                alwaysLinkToLastBuild: true,
                keepAll: true,
                reportDir: 'custom-report',
                reportFiles: '*.html',
                reportName: 'Custom Test Report'
            ])

            emailext(
                subject: "Jenkins Build: ${env.JOB_NAME} #${env.BUILD_NUMBER} -
${currentBuild.currentResult}",
                body: """
                    <h2>Playwright Test Execution Report</h2>
                    <p><b>Job:</b> ${env.JOB_NAME}</p>
                    <p><b>Build:</b> #${env.BUILD_NUMBER}</p>
                    <p><b>Status:</b> ${currentBuild.currentResult}</p>
                    <p><b>Test Suite:</b> ${params.TEST_SUITE}</p>
                    <p><b>Browser:</b> ${params.BROWSER}</p>
                    <p>
                        <a href="${env.BUILD_URL}">Open Jenkins Build</a> |
                        <a href="${env.BUILD_URL}Playwright_20HTML_20Report">Open Playwright HTML
Report</a> |
                        <a href="${env.BUILD_URL}Custom_20Test_20Report">Open Custom Report</a>
                    </p>
                """,
                to: "pavanoltraining@gmail.com",
                mimeType: "text/html"
            )
        }

        success {
```

```
            script {
                powershell 'Remove-Item -Recurse -Force allure-results'
            }
        }
    }
}
```

Jenkinsfile (Mac OS)

```
pipeline {
    agent any

    tools {
        git 'Default'
    }

    options {
        timestamps()
    }

    parameters {
        choice(
            name: 'TEST_SUITE',
            choices: [
                'test:master',
                'test:sanity',
                'test:regression',
                'test:e2e',
                'test:api',
                'test:web',
                'test:datadriven'
            ],
            description: 'Select the test suite to run'
        )
        choice(
            name: 'BROWSER',
            choices: [
                'chromium',
                'firefox',
                'webkit'
            ],
            description: 'Select the browser for tests'
        )
        choice(
            name: 'MODE',
            choices: [
                'headless',
                'headed'
            ],
            description: 'Run tests in headless or headed mode'
        )
    }

    environment {
        NODE_ENV = 'test'
        CI = '1'
```

```
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Install Dependencies') {
            steps {
                sh 'npm ci'
            }
        }

        stage('Install Playwright Browsers') {
            steps {
                sh 'npx playwright install'
                sh '''
                    if [ ! -f .env ] && [ -f .env.example ]; then
                        cp .env.example .env
                    fi
                '''
            }
        }

        stage('Run Playwright Tests') {
            steps {
                script {
                    def suite = params.TEST_SUITE
                    def browserFlag = "--project=${params.BROWSER}"
                    def modeFlag = params.MODE == 'headed' ? '--headed' : ''
                    sh "npm run ${suite} -- ${browserFlag} ${modeFlag}"
                }
            }
        }
    }

    post {
        always {
            archiveArtifacts(
                artifacts: 'reports/**,custom-report/**,allure-results/**',
                allowEmptyArchive: true
            )

            junit(
                testResults: 'reports/results.xml',
                allowEmptyResults: true
            )

            allure(
                includeProperties: false,
                results: [
                    [path: 'allure-results']
                ]
            )

            publishHTML([
```

```
                allowMissing: true,
                alwaysLinkToLastBuild: true,
                keepAll: true,
                reportDir: 'reports',
                reportFiles: 'index.html',
                reportName: 'Playwright HTML Report'
            ])

            publishHTML([
                allowMissing: true,
                alwaysLinkToLastBuild: true,
                keepAll: true,
                reportDir: 'custom-report',
                reportFiles: '*.html',
                reportName: 'Custom Test Report'
            ])

            emailext(
                subject: "Jenkins Build: ${env.JOB_NAME} #${env.BUILD_NUMBER} -
${currentBuild.currentResult}",
                body: """
                    <h2>Playwright Test Execution Report</h2>
                    <p><b>Job:</b> ${env.JOB_NAME}</p>
                    <p><b>Build:</b> #${env.BUILD_NUMBER}</p>
                    <p><b>Status:</b> ${currentBuild.currentResult}</p>
                    <p><b>Test Suite:</b> ${params.TEST_SUITE}</p>
                    <p><b>Browser:</b> ${params.BROWSER}</p>
                    <p><b>Mode:</b> ${params.MODE}</p>
                    <p>
                        <a href="${env.BUILD_URL}">
                            Open Jenkins Build
                        </a>
                        |
                        <a href="${env.BUILD_URL}Playwright_20HTML_20Report">
                            Open Playwright HTML Report
                        </a>
                        |
                        <a href="${env.BUILD_URL}Custom_20Test_20Report">
                            Open Custom Report
                        </a>
                    </p>
                """,
                to: "pavanoltraining@gmail.com",
                mimeType: "text/html"
            )
        }

        success {
            sh '''
                if [ -d allure-results ]; then
                    rm -rf allure-results
                fi
            '''
        }
    }
}
```